Fruit Classification

Data Science Project Documentation

Zaid Shawakfeh

Computer Engineering — German Jordanian University

March 2026

Supervisor: Gustav Willig

Table of Contents

Table of Contents 2
1. Problem Statement 3
2. Data Preprocessing 3
  2.1 Step-by-Step Preprocessing 3
    Step 1 — Removing Missing Values 3
    Step 2 — Lowercasing and Stripping Whitespace 3
    Step 3 — Removing Stray Digits from Text Columns 3
    Step 4 — Fuzzy Matching for Typo Correction 3
    Step 5 — Per-Fruit Outlier Removal (IQR Method) 4
    Step 6 — Data Type Enforcement and Column Renaming 4
  2.2 Feature Encoding for Model Input 4
3. Model Overview 5
  3.1 Decision Tree 5
  3.2 Random Forest 5
    Why Random Forest Outperforms a Single Decision Tree 5
  3.3 Logistic Regression 6
4. Hyperparameter Tuning: GridSearchCV + K-Fold Cross Validation 7
  4.1 Stratified K-Fold Cross Validation 7
  4.2 GridSearchCV 7
  4.3 Overfitting Detection 7
5. Model Results 8
  5.1 Decision Tree 8
  5.2 Random Forest 8
  5.3 Logistic Regression 10
6. Model Comparison 11
  6.1 Overall Performance 11
  6.2 Per-Class Analysis 11
  6.3 Recommendation 12
7. Advantages and Disadvantages of Each Model 13
  7.1 Decision Tree 13
  7.2 Random Forest 13
  7.3 Logistic Regression 13
8. Conclusion 13

1. Problem Statement

The goal of this project is to develop a machine learning classifier capable of distinguishing between three types of fruit: apples, bananas, and grapes. This is a supervised multi-class classification problem where the model must learn to predict the correct fruit type (label) based on a set of measurable features.

The dataset contains 200 records, each describing a fruit through four attributes: fruit type, color, size, and weight. The fruit type column serves as the target label, while color, size, and weight act as the input features for prediction.

Three classification algorithms were evaluated: Decision Tree, Random Forest, and Logistic Regression. Each model was tuned using GridSearchCV combined with Stratified K-Fold Cross Validation, and evaluated using accuracy, F1 score (weighted and macro), ROC-AUC, and a full classification report. The project was implemented entirely in Python within a Jupyter Notebook.

2. Data Preprocessing

Before training any model, the raw dataset required significant cleaning and standardization. The preprocessing pipeline was designed to ensure data quality, consistency, and model compatibility. All steps were implemented in a single Jupyter Notebook cell for full reproducibility and traceability.

2.1 Step-by-Step Preprocessing

Step 1 — Removing Missing Values

The first step was to remove any row containing at least one missing or null value using pandas dropna(). This ensures that no incomplete records propagate into the model training phase, which could otherwise lead to errors or biased predictions.

```
df.dropna(inplace=True)
```

Step 2 — Lowercasing and Stripping Whitespace

All three textual columns (fruit_type, color, size) were converted to lowercase and stripped of leading or trailing whitespace. This prevents duplicate categories arising purely from inconsistent capitalisation, such as treating 'Yellow' and 'yellow' as two different colors.

```
df[col] = df[col].str.strip().str.lower()
```

Step 3 — Removing Stray Digits from Text Columns

Several entries in the color and size columns contained erroneous digits embedded within the text, for example 'Yellow1' instead of 'yellow'. A regular expression was applied to strip all numeric characters from those columns, restoring the intended string values.

```
df[col] = df[col].apply(lambda x: re.sub(r"\d+", "", str(x)).strip())
```

Step 4 — Fuzzy Matching for Typo Correction

Even after digit removal, typographical errors remained in some entries, such as 'largee' instead of 'large'. Python's difflib.get_close_matches() was used to compare each value against a predefined list of canonical valid values for each column. Any value with at least 75% string similarity to a valid entry was automatically corrected. This approach is robust enough to fix multi-character typos without requiring manual inspection of every record.

```
matches = get_close_matches(value, valid_list, n=1, cutoff=0.75)
```

Step 5 — Per-Fruit Outlier Removal (IQR Method)

Outliers in the weight column were identified and removed using the Interquartile Range (IQR) method. Critically, the IQR bounds were computed separately for each fruit type rather than across the entire dataset. This is because the weight ranges for apples, bananas, and grapes are fundamentally different — a weight that is normal for an apple would be an extreme outlier for a grape. Any row whose weight fell below Q1 - 1.5*IQR or above Q3 + 1.5*IQR for its respective fruit was removed. This step eliminated 7 outlier records, leaving 193 clean rows.

```
Q1  = df.groupby('fruit_type')['weight'].transform('quantile', 0.25)
Q3  = df.groupby('fruit_type')['weight'].transform('quantile', 0.75)
IQR = Q3 - Q1
mask = (df['weight'] >= Q1 - 1.5*IQR) & (df['weight'] <= Q3 + 1.5*IQR)
```

Step 6 — Data Type Enforcement and Column Renaming

After all cleaning steps, the three textual columns were explicitly cast to Python string type to guarantee consistency. The fruit_type column was renamed to label to reflect its role as the target output class for all models.

```
df[col] = df[col].astype(str)
df.rename(columns={"fruit_type": "label"}, inplace=True)
```

2.2 Feature Encoding for Model Input

Since machine learning models require numeric input, the categorical columns color and size were encoded using scikit-learn's LabelEncoder before being passed to any model. This transformation assigns an integer to each unique string category. The final feature set used by all three models consisted of three columns: color_enc, size_enc, and weight.

```
le_color = LabelEncoder()
le_size  = LabelEncoder()
df_model["color_enc"] = le_color.fit_transform(df_model["color"])
df_model["size_enc"]  = le_size.fit_transform(df_model["size"])
FEATURES = ["color_enc", "size_enc", "weight"]
```

3. Model Overview

3.1 Decision Tree

A Decision Tree is a non-parametric supervised learning algorithm that partitions the feature space through a series of recursive binary splits. At each node, the algorithm selects the feature and threshold that best separates the classes according to a chosen criterion such as Gini impurity or information gain (entropy). The process continues until a stopping condition is reached, such as maximum depth or minimum samples per leaf.

Decision Trees are highly interpretable and require minimal data preparation, but they are prone to overfitting when grown too deep. A single deep tree will memorise the training data, achieving near-perfect training accuracy while performing poorly on unseen data.

Key hyperparameters tuned:

- max_depth — Controls how deep the tree grows. Limits overfitting by preventing the tree from memorising individual samples.
- criterion — The splitting metric: gini (faster) or entropy (information gain).
- min_samples_split — Minimum number of samples required to split an internal node.
- min_samples_leaf — Minimum number of samples required at a leaf node. Acts as a regularizer.

3.2 Random Forest

Random Forest is an ensemble method that builds a large number of Decision Trees, each trained on a random bootstrap sample of the data and using a random subset of features at each split. The final prediction is determined by majority vote across all trees. This approach is known as bagging (Bootstrap Aggregating).

Why Random Forest Outperforms a Single Decision Tree

A single Decision Tree is highly sensitive to the specific training data it sees — a small change in the dataset can lead to a completely different tree structure. Random Forest overcomes this variance problem by averaging the predictions of many diverse trees. Because each tree is trained on a different random subset of data and features, the individual errors of each tree are largely uncorrelated, and averaging them reduces the overall error significantly.

The key advantages of Random Forest over a single Decision Tree are:

- Lower variance — averaging hundreds of trees smooths out the noise of individual trees.
- Robustness to overfitting — individual trees may overfit, but the ensemble generalises far better.
- Feature importance — Random Forest provides a reliable ranking of how much each feature contributes to predictions.
- Stability — results are consistent across different random seeds and data samples.

Key hyperparameters tuned:

- n_estimators — Number of trees to build. More trees = more stable, but slower.
- max_depth — Maximum depth of each individual tree.
- max_features — Number of features considered at each split. 'sqrt' is standard for classification.
- min_samples_split / min_samples_leaf — Same regularization role as in Decision Tree.

3.3 Logistic Regression

Despite its name, Logistic Regression is a linear classification algorithm. It models the probability of each class using the softmax function (for multi-class problems), fitting a linear decision boundary in the feature space. It is simple, fast, and interpretable, but fundamentally limited to linearly separable data.

Because Logistic Regression is sensitive to the scale of input features, a StandardScaler was applied within a scikit-learn Pipeline, ensuring the scaler was only ever fitted on training data and never leaked information from the test set.

Key hyperparameters tuned:

- C — Inverse regularization strength. Small C = strong regularization = simpler model. Large C = fits training data more closely.
- penalty — Type of regularization. L2 (Ridge) was used, which shrinks all coefficients.
- solver — Optimization algorithm. lbfgs and saga were both evaluated.
- multi_class='multinomial' — Enables proper multi-class probability estimation for all three fruit classes simultaneously.

4. Hyperparameter Tuning: GridSearchCV + K-Fold Cross Validation

4.1 Stratified K-Fold Cross Validation

K-Fold Cross Validation is a technique for evaluating model performance more reliably than a single train-test split. The training data is divided into K equal folds. The model is trained K times, each time using K-1 folds for training and the remaining fold as a validation set. The final performance metric is the average across all K validation folds.

Stratified K-Fold was specifically chosen because it preserves the original class proportions in each fold. This is important in a multi-class problem to ensure that no fold ends up with a disproportionate number of one fruit type, which could bias the validation results. Five folds (n_splits=5) were used throughout this project.

```
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

4.2 GridSearchCV

GridSearchCV automates the process of hyperparameter selection by exhaustively trying every combination of hyperparameter values defined in a parameter grid. For each combination, it performs K-Fold Cross Validation and computes the mean validation score. The combination with the highest mean validation F1 score (weighted) is selected as the best set of hyperparameters.

By passing the StratifiedKFold object directly to the cv parameter of GridSearchCV, both techniques are combined: the grid search handles hyperparameter enumeration while the stratified K-Fold handles robust evaluation of each candidate. The return_train_score=True flag additionally records training scores per fold, enabling direct overfitting diagnostics.

```
grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=kfold,
    scoring='f1_weighted',
    n_jobs=-1,
    return_train_score=True
)
```

4.3 Overfitting Detection

Overfitting was monitored through three complementary methods in each model cell. First, the mean train accuracy and mean test accuracy from the K-Fold cross-validation were compared — a gap larger than 0.10 triggers an overfit warning. Second, the hold-out test set accuracy (on data never seen during tuning) was compared to the training accuracy. Third, validation curves were plotted showing how train and test accuracy diverge as the key hyperparameter (max_depth for trees, C for logistic regression) increases — the point of divergence visually marks the onset of overfitting.

5. Model Results

5.1 Decision Tree

The Decision Tree was evaluated with a test size of 0.30 (70% training, 30% test). GridSearchCV selected entropy as the splitting criterion, a maximum depth of 7, a minimum leaf size of 4, and a minimum split size of 2.

K-Fold Cross Validation Results:

- Training Accuracy: 0.9275 ± 0.0049 — consistent and stable across folds.
- Test Accuracy: 0.8548 ± 0.0271 — solid generalisation with acceptable variance.
- Overfit Gap: 0.0727 — within the healthy threshold of 0.10.
- F1 Weighted: 0.8528 ± 0.0285 | F1 Macro: 0.8549 ± 0.0280

Hold-Out Test Set Results:

- Test Accuracy: 0.8621 | F1 Weighted: 0.8621 | F1 Macro: 0.8632
- ROC-AUC (OvR): 0.9555 — excellent discrimination between all three classes.
- Grapes were classified with perfect precision and recall (F1 = 1.00), confirming that weight alone makes grapes very easy to separate from the other fruits.
- Apples and bananas had similar F1 scores of 0.80 and 0.79, reflecting some overlap in their feature distributions.

```
Best hyperparameters: {'criterion': 'entropy', 'max_depth': 7, 'min_samples_leaf': 4, 'min_samples_split': 2}
Best CV F1 (validation) : 0.8658

========== Decision Tree — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.9275 ± 0.0049
  Test  Accuracy  : 0.8548 ± 0.0271
  Test  F1 (weighted): 0.8528 ± 0.0285
  Test  F1 (macro)   : 0.8549 ± 0.0280
  Overfit gap     : 0.0727  ✓ Healthy

========== Decision Tree — Hold-Out Test Set ==========
  Train Accuracy : 0.9333
  Test  Accuracy : 0.8621
  F1  (weighted) : 0.8621
  F1  (macro)    : 0.8632
  ROC-AUC (OvR)  : 0.9555

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.80      0.80      0.80        20
      banana       0.79      0.79      0.79        19
       grape       1.00      1.00      1.00        19

    accuracy                           0.86        58
   macro avg       0.86      0.86      0.86        58
weighted avg       0.86      0.86      0.86        58
```

Figure 1: Decision Tree — Evaluation Results (test_size=0.30)

5.2 Random Forest

The Random Forest was evaluated with a test size of 0.15 (85% training, 15% test). GridSearchCV selected max_depth=10, max_features='sqrt', min_samples_leaf=2, min_samples_split=5, and n_estimators=200.

K-Fold Cross Validation Results:

- Training Accuracy: 0.9430 ± 0.0074 — slightly higher train accuracy than the Decision Tree.
- Test Accuracy: 0.8394 ± 0.0096 — very low variance (±0.0096), indicating highly stable generalisation.
- Overfit Gap: 0.1036 — marginally above the 0.10 threshold, indicating a slight tendency toward overfitting.
- F1 Weighted: 0.8380 ± 0.0095 | F1 Macro: 0.8404 ± 0.0074

Hold-Out Test Set Results:

- Test Accuracy: 0.8205 | F1 Weighted: 0.8205 | F1 Macro: 0.8272
- ROC-AUC (OvR): 0.9378 — strong class separation capability.
- As with the Decision Tree, grapes were perfectly classified (F1 = 1.00). Apples and bananas both achieved F1 = 0.74.
- The slight overfit gap and lower hold-out accuracy compared to the Decision Tree suggest the smaller test size (0.15) left less data for final evaluation.

```
Best hyperparameters: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 200}
Best CV F1 (validation) : 0.8823

========== Random Forest — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.9430 ± 0.0074
  Test  Accuracy  : 0.8394 ± 0.0096
  Test  F1 (weighted): 0.8380 ± 0.0095
  Test  F1 (macro)   : 0.8404 ± 0.0074
  Overfit gap     : 0.1036  ⚠ Possible overfit

========== Random Forest — Hold-Out Test Set ==========
  Train Accuracy : 0.9481
  Test  Accuracy : 0.8205
  F1  (weighted) : 0.8205
  F1  (macro)    : 0.8272
  ROC-AUC (OvR)  : 0.9378

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.77      0.71      0.74        14
      banana       0.71      0.77      0.74        13
       grape       1.00      1.00      1.00        12

    accuracy                           0.82        39
   macro avg       0.83      0.83      0.83        39
weighted avg       0.82      0.82      0.82        39
```

Figure 2: Random Forest — Evaluation Results (test_size=0.15)

5.3 Logistic Regression

The Logistic Regression was evaluated with a test size of 0.15. GridSearchCV selected C=0.01, penalty='l2', and solver='lbfgs' — a strongly regularized model, which suggests the data is not linearly separable with the current features.

K-Fold Cross Validation Results:

- Training Accuracy: 0.7306 ± 0.0194 — noticeably lower than tree-based models.
- Test Accuracy: 0.7198 ± 0.0965 — high variance (±0.0965) indicates instability across folds.
- Overfit Gap: 0.0107 — practically zero, confirming no overfitting, but also suggesting the model underfits.
- F1 Weighted: 0.7088 ± 0.1007 | F1 Macro: 0.7134 ± 0.0967

Hold-Out Test Set Results:

- Test Accuracy: 0.6410 | F1 Weighted: 0.6124 | F1 Macro: 0.6171
- ROC-AUC (OvR): 0.8193 — still reasonable discriminative ability at the probability level.
- Banana classification was particularly poor (F1 = 0.38, recall = 0.31), indicating the linear decision boundary struggles to reliably separate bananas from other fruits.
- Grapes remained the easiest class (F1 = 0.83), though no longer perfectly classified as with the tree models.

```
Best hyperparameters: {'model__C': 0.01, 'model__penalty': 'l2', 'model__solver': 'lbfgs'}
Best CV F1 (validation) : 0.7359

========== Logistic Regression — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.7306 ± 0.0194
  Test  Accuracy  : 0.7198 ± 0.0965
  Test  F1 (weighted): 0.7088 ± 0.1007
  Test  F1 (macro)   : 0.7134 ± 0.0967
  Overfit gap     : 0.0107  ✓ Healthy

========== Logistic Regression — Hold-Out Test Set ==========
  Train Accuracy : 0.7532
  Test  Accuracy : 0.6410
  F1  (weighted) : 0.6124
  F1  (macro)    : 0.6171
  ROC-AUC (OvR)  : 0.8193

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.64      0.64      0.64        14
      banana       0.50      0.31      0.38        13
       grape       0.71      1.00      0.83        12

    accuracy                           0.64        39
   macro avg       0.62      0.65      0.62        39
weighted avg       0.61      0.64      0.61        39
```

Figure 3: Logistic Regression — Evaluation Results (test_size=0.15)

6. Model Comparison

The table below summarises all key evaluation metrics across the three models for direct comparison.

Metric	Decision Tree	Random Forest	Logistic Regr.
Best Test Size	0.30	0.15	0.15
Best CV F1	0.8658	0.8823	0.7359
KFold Train Acc	0.9275 ± 0.0049	0.9430 ± 0.0074	0.7306 ± 0.0194
KFold Test Acc	0.8548 ± 0.0271	0.8394 ± 0.0096	0.7198 ± 0.0965
Overfit Gap	0.0727 ✓	0.1036 ⚠	0.0107 ✓
Hold-Out Test Acc	0.8621	0.8205	0.6410
F1 Weighted	0.8621	0.8205	0.6124
F1 Macro	0.8632	0.8272	0.6171
ROC-AUC (OvR)	0.9555	0.9378	0.8193
Grape F1	1.00	1.00	0.83
Apple F1	0.80	0.74	0.64
Banana F1	0.79	0.74	0.38

6.1 Overall Performance

The Decision Tree achieved the highest hold-out test accuracy (0.8621) and the highest ROC-AUC score (0.9555), making it the best-performing model in this project. This is partly attributable to its larger test size of 0.30, which provided a more representative evaluation sample. Its overfit gap of 0.0727 remained well within the healthy threshold.

The Random Forest achieved a slightly lower hold-out accuracy (0.8205) but demonstrated the most stable K-Fold test accuracy (±0.0096 standard deviation), confirming its strength as a robust ensemble method. Its marginally elevated overfit gap (0.1036) was the only concern, and it can be addressed by tuning min_samples_leaf or reducing n_estimators.

Logistic Regression performed significantly worse across all metrics, with a hold-out accuracy of only 0.6410 and an F1 weighted score of 0.6124. The high K-Fold variance (±0.0965) and the very low C value selected by GridSearchCV (C=0.01) both strongly suggest that the feature space is not linearly separable, and Logistic Regression is not an appropriate model for this dataset without additional feature engineering such as polynomial features or non-linear transformations.

6.2 Per-Class Analysis

Grapes were perfectly classified by both tree-based models (F1 = 1.00) and very well classified by Logistic Regression (F1 = 0.83). This is expected because grapes are significantly lighter than apples and bananas, making weight a highly discriminative feature that even a linear model can leverage.

Apples and bananas presented the main challenge for all models due to overlapping weight ranges and similar color and size distributions. The tree-based models handled this overlap well through non-linear splits, while Logistic Regression — limited to a linear boundary — struggled significantly, especially for bananas where recall dropped to only 0.31.

6.3 Recommendation

Based on all evaluation metrics, the Decision Tree is recommended as the primary model for this classification task, offering the best balance of accuracy, interpretability, and generalisation. The Random Forest is recommended as a secondary model where prediction stability and feature importance analysis are priorities. Logistic Regression is not recommended in its current form, but could be revisited after polynomial feature expansion or dimensionality augmentation.

7. Advantages and Disadvantages of Each Model

7.1 Decision Tree

Advantages:

- Highly interpretable — the tree structure can be visualised and explained to non-technical stakeholders.
- No feature scaling required — works directly with raw numeric values.
- Handles both linear and non-linear decision boundaries.
- Fast to train and predict.

Disadvantages:

- Prone to overfitting if max_depth is not constrained.
- Unstable — small changes in training data can produce very different trees.
- Greedy splits do not guarantee a globally optimal tree.

7.2 Random Forest

Advantages:

- Much more robust to overfitting than a single Decision Tree.
- Provides reliable feature importance scores.
- Handles high-dimensional data well.
- Highly stable — averaging hundreds of trees reduces prediction variance dramatically.

Disadvantages:

- Less interpretable than a single Decision Tree — it is a black-box ensemble.
- Significantly slower to train due to the number of trees.
- Higher memory consumption.
- Can still overfit on noisy datasets if not properly regularized.

7.3 Logistic Regression

Advantages:

- Extremely fast to train and predict.
- Fully interpretable — coefficients directly show the influence of each feature.
- Provides well-calibrated class probabilities.
- Very low risk of overfitting due to built-in regularization (L2).

Disadvantages:

- Assumes a linear decision boundary — fails when classes are not linearly separable.
- Requires feature scaling for reliable convergence.
- Sensitive to multicollinearity between features.
- Cannot capture interaction effects between features without manual engineering.

8. Conclusion

This project successfully developed and evaluated three classification models for distinguishing between apples, bananas, and grapes using color, size, and weight as input features. A thorough preprocessing pipeline was implemented to handle missing values, textual inconsistencies, digit contamination, typographical errors, and per-fruit weight outliers.

All three models were tuned using GridSearchCV with Stratified K-Fold Cross Validation, ensuring that hyperparameter selection was based on robust multi-fold evaluation rather than a single split. Overfitting was monitored through the train-test accuracy gap, cross-validation variance, and visual validation curves.

The Decision Tree achieved the best overall performance with a hold-out test accuracy of 86.21% and a ROC-AUC of 0.9555, closely followed by Random Forest at 82.05% accuracy and ROC-AUC of 0.9378. Logistic Regression underperformed at 64.10% test accuracy, confirming that the fruit classification problem requires a non-linear model to be solved effectively.

Grapes were consistently the easiest class to classify across all models due to their distinctly lower weight. Apples and bananas remain the primary source of misclassification, and future work could focus on adding additional distinguishing features or applying more sophisticated ensemble methods to further improve performance on these two classes.